

GPTIMER

GPTIMER

- 1. GPTIMER Timer Overview
- 2. Core Concepts
 - 2.1 Hardware Timer Architecture
 - 2.2 `IRAM_ATTR` Attribute
 - 2.3 `esp_timer` vs GPTIMER Comparison
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 Global Variables and Timer Handle
 - 5.2 Timer ISR
 - 5.3 `setup()` — Four Steps of Timer Initialization
- 6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Normal Behavior
 - 6.3 Verification Method
- 7. Common Issues

1. GPTIMER Timer Overview

GPTIMER directly operates ESP32 hardware timers, executing callbacks in ISR with deterministic delay and no RTOS scheduling jitter. This experiment configures a 1-second period hardware timer and prints the trigger count in the ISR. Compared to `esp_timer` in the previous section, GPTIMER trades convenience (ISR cannot use Serial, this experiment is for teaching only) for nanosecond-level timing determinism.

Typical Application Scenarios:

Application Type	Example
Precision Timing	Motor PWM deadband control, stepper motor pulse sequences
Frequency Measurement	Input capture frequency measurement, tachometer signal processing
Pulse Generation	Precision delay triggering, laser/ultrasonic ranging trigger signals
High-speed Counting	Encoder pulse counting, production line photoelectric counting

2. Core Concepts

2.1 Hardware Timer Architecture

ESP32 has 4 groups of 64-bit hardware timers (TIMG0/1, 2 per group). The clock source is APB_CLK = 80MHz, divided to obtain the counting frequency.

```
timerBegin(divider) Parameter Description:
  APB_CLK = 80,000,000 Hz
  Post-divider frequency = 80,000,000 / divider
  If divider = 80 → post-divider = 1MHz → each tick = 1µs

timerAlarm(timer, 1000000, true, 0) Parameter Description:
  1000000 = alarm threshold (tick count)
  At 1MHz, 1,000,000 ticks = 1,000,000 µs = 1 second
  true = auto-reload, counter resets to zero after trigger and restarts counting
```

2.2 IRAM_ATTR Attribute

```
void IRAM_ATTR Timer_ISR(void) { ... }
```

`IRAM_ATTR` places function code in SRAM instead of Flash. Flash reads have the risk of cache misses, and a cache miss in ISR can cause unpredictable delays or even watchdog reset. `IRAM_ATTR` ensures ISR code is always in RAM with deterministic execution time.

This experiment calls `Serial.printf` in the ISR - this is **strictly prohibited** in production code. It is only used here for teaching convenience to observe timer behavior.

2.3 esp_timer vs GPTIMER Comparison

Feature	esp_timer	GPTIMER
API Style	<code>esp_timer_create</code> + <code>start_periodic</code>	<code>timerBegin</code> + <code>timerAttachInterrupt</code> + <code>timerAlarm</code> + <code>timerStart</code>
Callback Context	Timer Task (Serial available)	ISR (Serial unavailable)
Timing Jitter	Has RTOS scheduling jitter	Nanosecond-level determinism
<code>volatile</code> Required	Yes (cross-task sharing)	Yes (ISR and main loop sharing)

3. Hardware Dependencies

This project is a pure software experiment with no external hardware required. Only USB cable needed to connect serial port for output viewing.

4. Project Architecture and Flow

```
setup()
├ Serial.begin(115200)           // Initialize serial port
├ timer = timerBegin(1000000)    // Initialize timer, divide to 1MHz
├ timerAttachInterrupt(timer, ISR) // Bind ISR function
├ timerAlarm(timer, 1000000, true, 0) // Alarm threshold = 1s, auto-reload
└ timerStart(timer)             // Start counting

loop()                          // Empty loop

[Hardware Interrupt]
└ Timer_ISR()                  // Triggers once per second
  ├── trigger_count++          // Increment counter
  └ Serial.printf(...)         // Print (teaching only! Forbidden in production)
```

5. Source Code Details

Full source code: [Source Code](#) → [Arduino](#) → [1.Basic Peripherals](#) → [5.GP_TIMER](#) → [5.GP_TIMER.ino](#)

5.1 Global Variables and Timer Handle

`hw_timer_t*` is a pointer to the hardware timer control block, `volatile` variables are used for sharing between ISR and main loop.

```
hw_timer_t *timer = NULL;
volatile uint32_t trigger_count = 0;
```

`timer` does not need `volatile` - it is initialized in `setup()` and never changes afterward, ISR does not write to it.

5.2 Timer ISR

`IRAM_ATTR` places the function in RAM ensuring deterministic ISR execution time.

```
void IRAM_ATTR Timer_ISR(void) {
    trigger_count++;
    Serial.printf("Trigger Count: %u | Current Time: %lu ms\n", trigger_count, millis());
}
```

Warning: `Serial.printf` is unsafe in ISR, this experiment uses it only for teaching convenience. In real projects, ISR should only set flags, printing should be done in `loop()`.

5.3 `setup()` — Four Steps of Timer Initialization

Hardware timer initialization has more steps than `esp_timer`, each step corresponds to an independent API.

```
void setup() {
  Serial.begin(115200);
  Serial.println("GPTIMER Pure Timer Test Started");

  timer = timerBegin(1000000);           // ① Initialize, divide to 1MHz
  timerAttachInterrupt(timer, Timer_ISR); // ② Bind ISR
  timerAlarm(timer, 1000000, true, 0);   // ③ Alarm threshold 1s, auto-reload,
infinite
  timerStart(timer);                     // ④ Start
}

void loop() {} // Empty loop - all work done by timer ISR
```

The four steps must not be reordered: timer must be created first, then bind ISR, set alarm threshold, finally start.

6. Operation Flow

6.1 Build and Flash

Arduino flash process: See `Arduino → 1. Before Development → 2. Build and Flash`.

6.2 Normal Behavior

Open **Serial Monitor** (115200 bps), print one line of trigger count every second.

```
=====
GPTIMER Pure Timer Test Started
Timer Period: 1000ms (1s)
=====
Trigger Count: 1 | Current Time: 1037 ms
Trigger Count: 2 | Current Time: 2037 ms
Trigger Count: 3 | Current Time: 3037 ms
Trigger Count: 4 | Current Time: 4037 ms
Trigger Count: 5 | Current Time: 5037 ms
Trigger Count: 6 | Current Time: 6037 ms
Trigger Count: 7 | Current Time: 7037 ms
Trigger Count: 8 | Current Time: 8037 ms
Trigger Count: 9 | Current Time: 9037 ms
```

6.3 Verification Method

Operation	Expected Result
Power on and observe serial	Print one line per second, count increments
Stopwatch comparison	10 triggers ≈ 10 seconds

7. Common Issues

Symptom	Cause	Solution
Serial hangs in ISR	<code>Serial</code> operations forbidden in ISR	Change to only set flags, print in <code>loop()</code>
Timing inaccurate	Divider configuration error	Confirm <code>APB_CLK / divider = 1MHz</code>
Compilation error <code>hw_timer_t</code> undefined	Missing ESP32 Arduino Core headers	Confirm board selected is ESP32S3